High-level performance analysis and troubleshooting of a Postgres node

Goals and methodologies

In this runbook, we define how to analyze performance and general health of a single Postgres node to answer the questions quickly (within one minute):

1. Is this Postgres node experiencing issues?
2. If yes, then in which areas these issues are? (In other words, what areas are worth researching further?)

We will be relying on these two methodologies:

1. Wait event analysis a.k.a. Active Session History, or ASH (an analog to Performance Insights in AWS RDS or Query Insights in GCP CloudSQL).
2. "Four Golden Signals" (see Google SRE book): Latency, Traffic, Errors, Saturation with respect to a single Postgres node.
   - high level (TPS, query processing latency)
   - physical level (CPU, disk IO, etc.)
   - at Postgres component level (e.g., replication or WAL archiving lag)

Dashboards to be used

1. Postgres node performance overview (high-level)

Additionally, for further steps:

1. Postgres aggregated query performance analysis
2. Postgres single query performance analysis
3. Postgres wait events analysis

Analysis steps

Step 1. Fastest check using just one panel – ASH

Check the first panel "Active session history (ASH)", often, it help identify issues and directions for further research very quickly, just looking at a single chart (:warning: Although, do not skip all the further steps! It is worth to spend a minute for shallow but very wide analysis, not to miss important facts).

The ASH panel shows the number of wait events in time, basically it represents what database is busy with. It has reference line with number of vCPUs on a node.

- If you see "CPU or Uncategorized wait event (green) + LWLock wait events (magenta)" are above the vCPUs count (grey dotted CPU reference line), we potentially have an issue with CPU consumption by Postgres. To verify this, please refer to the next section in this runbook.

- If IO wait events (dark blue) level is much higher than usual level observed, we potentially have an issue with IO consumption by Postgres. To verify this please, refer to the next step in this runbook.
- If Lock wait events (purple) level is much higher than usual level, we potentially have issue with heavy locks (relation- and row-level locks). To verify this, please refer to the next step in this runbook.

To review Wait events in more details and from various point of views, please use "Wait events analysis dashboard" (WIP).

Step 2. From ASH down to Host stats

Verify problem symptoms observed on the ASH panel by reviewing the Host Stats section:

- CPU:
  - "CPU usage" and "Load average" panels to understand CPU consumption on the host.
  - "User vs. System time from pg_stat_kcache" to review CPU time consumed by queries. Work with Postgres single query performance analysis to review main contributors to CPU consumption.
- IO:
  - Disk read and write latency panel and Disk read and write throughput to understand IO consumption and compare with other periods;
  - Shared block read, write and hits from pg_stat_statements and physical reads and writes from pg_stat_kcache panels to review IO generated by queries. Work with Postgres single query performance analysis to review main contributors to IO consumption.

Step 3. 30,000 ft view at SQL workload and Postgres components

Check correlation between Postgres performance issues and Postgres stats:

- **Latency**:
  - "Statements time per call" shows average execution end plan (query latency); if you observe a spike on query latency, work with Postgres single query performance analysis to find main contributors to the increased latency.
- **Traffic** – there are many ways to look at it:
  - "Calls (by pg_stat_statements)" (a.k.a. QPS) and "Transactions" (a.k.a. TPS) panels are the key throughput characteristics of the workload. The high-level dashboard show overall values, for further analysis, use Postgres single query performance analysis that provides QPS segmented by pg_stat_statements' `queryid`.
  - "Statements total time (by pg_stat_statements)" (a.k.a. DB time) - shows total time spent by database serving (planning and executing) SQL queries. This is a very interesting metric, measured in "seconds per second"; it can be viewed as amount of time, number of seconds,

the database server spends each second. For example, if 10 seconds are spend per a second, it means 10 backends are needed to handle this workload (not necessarily this translates to CPU usage: e.g., some backend might be waiting on a lock acquisition).
  - Shared blocks read/write/hits (by pg_stat_statemetns) and physical reads/writes (by pg_stat_kcache) panels show IO activity by queries, both at the buffer pool (shared buffers) and physical disk levels. If there are anomalies in these areas, proceed to Postgres single query performance analysis to study the corresponding metrics further, with segemnentation by pg_stat_statements' `queryid`.
  - WALs panels show how much WAL is being generared by current workload and autovacuum workers. Increased rate potentially lead to increased disk IO (review respective host stats panels above), and might also lead to high replication and WAL archiving lag (review respective panels below).
- **Errors**:
  - "Commits vs. Rollbacks" and "Commit ratio" panels visualize how many transactions end up with ROLLBACKs compared to COMMITs. Higher ROLLBACK rates need to be studied using Kibana // TODO: useful links here.
  - "Errors from Postgres logs" show the number of errors registeded in Postgres logs. These errors may be from SQL queries, but also from various Postgres components (e.g., replication), not necessarily related to SQL query processing. The next step here is to study logs in Kibana // TODO links
- **Saturation**:
  - The main tool for saturation risk analysis used at GitLab: Tamland; the "Postgres node performance overview (high-level)" dashboard discussed here can be used for additional analysis of Postgres workload and components, for a single Postgres node.
  - Host resources: CPU and disk IO – if saturated, the next steps:
    * Postgres single query performance analysis, particularly pg_stat_kcache panels to see if some queries contribute to the saturation the most; these queries need optimization (reduction of latency, or QPS, or both).
    * If no parts of query workload are identified as significantly contributing to the saturation, then next step is to study ASH (wait events) and use BPF tooling to see if some Postgres components or non-Postgres processes are playing key role in the saturation.
  - Connection saturation:
    * Hard limit: `max_connections` observed on panel "Backends by state" (in Postgres, "connection", "session", and "backend" and interchangeable terms) – normally, we are far from achieving it, pgBouncer helps; if in risk or saturated, analyze connections by state ("Non-idle backends by state").
    * Soft limit: number of non-idle connections observed on panel

"Non-idle backends by state" (including idle-in-transaction but excluding idle connections) – normally, we don't want to reach / significantly exceed the number of vCPUs the host has. If it happens, again, analyze connections by state.
    - Next step: apply query analysis using Postgres single query performance analysis.
  - Other types of saturation are possible, for example related to individual Postgres components (replication-related walsender, walreceiver, logical replication worker; autovacuum; checkpointer and bgwriter; WAL archiver; buffer pool). // TODO: clarify quick review of these areas once the corresponding panels are finished – WIP

5. As noted multiple times, in many cases, once specific areas of workload or Postgres components that look potentially in trouble are identified, the natural next step is to work with Postgres single query performance analysis, that provides detailed analysis of SQL query workload based on segmentation by `queryid` and numerous metrics from pg_stat_statements, pg_stat_kcache, and pg_wait_sampling. It has both table and chart forms of Top-N lists of queries for each metric, plus corresponding derivatives (e.g. query execution time per second is DB time segmented by `queryid`, and derivative from it, execution time per query is average latency for particular `queryid`).

6. And the next step in many cases, is reviewiing behavior of individual part of workload – particular `queryid` – using Postgres single query performance analysis.

Useful links

- Four Golden Signals (Google SRE book)
- pg_wait_sampling extension
- // TODO: add links to additional runbooks such as "Locking issue troubleshooting runbook"